



A sound response to long-tailed changes in business process management

Hongmei Cao^{1,2,3} · Xi Chen^{1,2,3} · Liang Zhang^{1,2,3} · Tiange Zhang^{1,2,3} · Xiaochun Xiao^{1,2,3}

Received: 26 October 2020 / Revised: 25 May 2022 / Accepted: 28 June 2022 / Published online: 18 August 2022
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2022

Abstract

Enterprises rely on their process-aware information system (PAIS) to conduct business. Therefore, appropriately responding to environmental changes is vital for enterprises to maintain competitiveness. However, one type of change, namely the long-tailed change (LTC), has been overlooked by traditional business process management practice because of its variety and infrequency. Just as the long-tailed effect reveals, the impact of LTC on enterprise PAIS might be no less dramatic than the impact of high-frequency changes. Since business process models are core assets of an enterprise, it is profitable to reuse them efficiently while tackling the conflict of flexibility and applicability in a timely way. This paper proposes a process model maintenance approach to responding to LTCs. By supporting business analysts to add syntax-correct annotations to existing business process models, the approach achieves an agile, error-free, and low-cost mechanism for dealing with LTCs.

Keywords Process-aware information system (PAIS) · Long-tailed changes (LTCs) · Business process management (BPM) · Syntax-directed editor generator · Abstract grammar tree (AST)

1 Introduction

Most companies, government offices, and organizations (Hereafter referred to as the organizations) require a sequence of activities in their operations to achieve a specific goal. Usually, these activities are coordinated by a process-aware information system (PAIS) in the light of the business process management (BPM) principles [1].

As the new economy comes, intense competition forces enterprises to continuously adapt to the ever-changing operating environment to handle uncertainties or capture emerging opportunities. How to respond to changes in a timely and proactive manner turns out to be fundamental for enterprises to maintain their core competencies in the new economy era. Among these changes, some occur frequently, are captured by stakeholders, and are modeled as business processes. In contrast, stakeholders overlook other changes with a wide range but low frequency due to business or technical limits. However, as indicated by the long-tail theory, cumulative effects in the long-tail region will be as significant as those in the short-head area. Therefore, for BPM, the need to respond to long-tail changes is growing more vital in the ever-changing social and economic environment. In this paper, the long-tailed change (LTC) means those change requirements with great variety and sporadic with low frequency but is significant to the business. These characteristics make it difficult for firms to adapt to this part of the variation through conventional change management approaches in BPM.

As we know, business process models are the core assets of an enterprise. Usually, they reflect unique knowledge, which implies that reusing business process models in change management is an efficient strategy to reduce costs, maintain quality, and respond agilely to changes. As a result, it is worth

✉ Liang Zhang
lzhang@fudan.edu.cn

Hongmei Cao
hmcao18@fudan.edu.cn

Xi Chen
chenxi17@fudan.edu.cn

Tiange Zhang
ztiane@fudan.edu.cn

Xiaochun Xiao
xxiaochun@fudan.edu.cn

¹ School of Computer Science, Fudan University, Shanghai, China

² Shanghai Key Laboratory of Data Science, Shanghai, China

³ Shanghai Institute of Intelligent Electronics and Systems, Shanghai, China

exploring the possibility of balancing the conflict of maximizing the reuse of existing process models and the capability of adaptation to environmental changes promptly.

In response to the above-mentioned issues, this paper proposes an approach to maintaining process models in response to LTCs. It implements a set of agile, lightweight, and low-cost LTC resilience mechanisms that support domain experts change the behavior of legacy models by attaching annotations. These annotations include IFTTT rules [2] to formulate functional change and value-exchange rules to describe non-functional requirements. To facilitate domain experts to deal with various situations of LTC, we also provide a syntax-directed editor generator based on the abstract syntax tree (AST), which guarantees annotation mechanisms are correct and usable. A enriched version of the open-source process engine Activiti¹ interprets these annotations. It is worth noting that due to the characteristics of LTC, it is easy to resume ordinary business by removing some or all these annotations.

All in all, the main technical contributions of this paper are many folds,

- Identifying the need for an important class of change management in BPM, i.e., the long-tailed changes with process models that previous approaches had overlooked;
- Proposing an agile approach to change management in response to LTCs, allowing domain experts to change the behavior of processes without the help of IT staff;
- Developing a syntax-directed editor generator designed to improve the usability for domain experts.

The rest of the paper is organized as follows. First, we discussion related work in Sect. 2 to clarify our motivation, and then skeleton the system architecture in Sect. 3. Next, we deliberate on various techniques, i.e., annotation modeling in Sect. 4 and message routine mechanism in Sect. 5. In Sect. 6, we discuss design decision with some real scenarios, it provides necessary information for case studies in Sect. 7. Finally, we conclude our study and outlook future directions.

2 Related work

This section summarizes related work in change management and domain-specific language application in BPM to clarify our motivation.

2.1 Change management in BPM

Change management is a hot topic in BPM. There has been plenty of studies on the subject, e.g., [3–5], ranging from model level (static changes) to instance level (dynamic

changes). Two systematic studies by Reichert et al. [6] and Song et al. [7] are worth highlighting. They provide comprehensive approaches to the change management problem in BPM. In contrast with both problems and solutions they mentioned, our work identifies a new type of change, i.e., LTC, and the adaption-by-annotation solution is more suitable for the LTC scenarios. This feature also makes our solution superior to similar annotation-based approaches, e.g., [8] by Intrigila et al., where they merely focus on data aspects.

Unlike most techniques to change management at the install level, including our previous rule-based dynamical adaptation framework [9], the solution in this study extends the adaptation capabilities from functions to business values, which is more coherent to BPM objectives [1].

2.2 Domain specific language

Domain-specific languages (DSLs) [10] have been studied for decades and can be considered a mature technology in a general sense. For example, domain-specific data structure in [8] and DSL in [11, 12] are contemporary schemes of DSL for BPM modeling and adaptation. However, if the domain or situation changes steadily, the DSL has to be adjusted repeatedly to catch pace with the change. It lays an obstacle for all the above solutions in the context of LTC. This paper proposes a solution to address the problem that provides domain experts with adjustable vocabulary, a customizable grammar defining tool, and an automatic editor generator. Combined with the annotation mechanism, this DSL toolkit benefits domain experts to handle LTC accordingly.

3 System architecture

Our goal is to set up a scheme that enables domain experts to reuse process models in the context of LTC by attaching easy annotations and helps enterprises maintain business process models efficiently by refining the L2L framework [13]. We sketch the scope as shaded components in Fig. 1.

Here, annotations were chosen as a medium to enable domain experts to control the behavior of business processes. There are two types of annotations,

- Annotations in IFTTT rules that intervene processes' execution paths, and
- The valueExchange annotations that declare and monitor business value proposition and maintain constraints in value exchange.

As illustrated in Fig. 1 An *IFTTT interpreter* parses the two types of annotations authored by a *Web-style editor* that is customized to the LTC scenario by a *Web Editor Generator*. The generator accepts the input of grammar

¹ <https://www.activiti.org>.

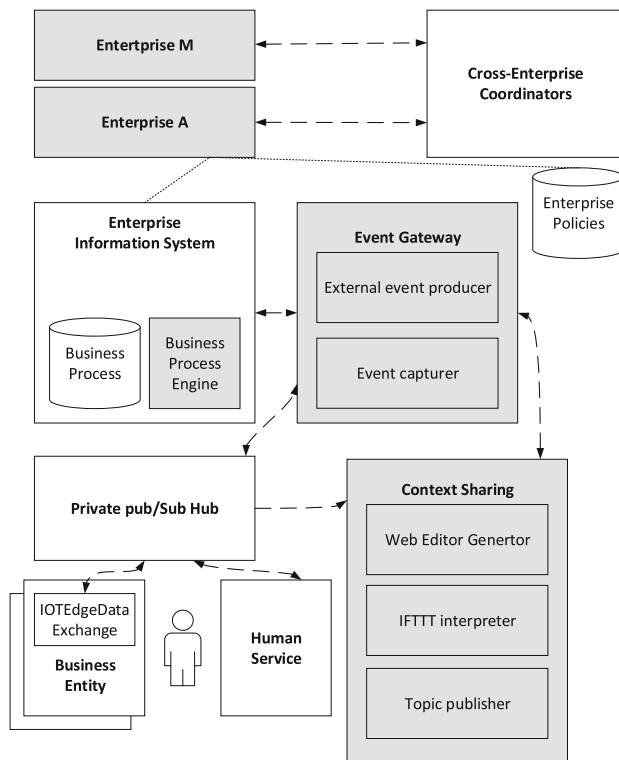


Fig. 1 Adopt the L2L [13] framework by refining shaded components that support annotation-based process adaptation and facilitate collaboration across PAIS

rules from architects and produces a corresponding editor for business operators to write syntax-correct annotations. An enriched *Business Process Engine* interprets annotations against messages from the *Event Capture* and communicates with other PAIS via the *Topic Publisher* with messages from the *External Event Producer*. In this way, we achieve the agile adaptation to LTC and reuse legacy business process models to the greatest extent [13].

In the following sections, we will elaborate on these shaded components.

4 Annotation modeling

In BPMN, annotations are uninterpreted comments in free forms. We leverage the annotation mechanism to incorporate functional and non-functional modifiers to adjust process models and control their communication according to the specification of domain experts.

4.1 Basic properties of the annotation

We choose annotations as an adaptation mechanism due to their non-intrusive nature and convenience. By non-intrusive, we mean that the same process model with annotations has no impact on most organizations' business except the PAIS facing the LTC issues. And, as a decoration style, annotation is more comfortable to business operators if we provide them a suitable language and some measures for correctness.

Annotations consist of basic property definitions that can be extended further. We found that this design is useful in different application scenarios. For example, we can give the basic definitions in Table 1 for one of our case studies.

- *Id* stands for a universally unique identifier for each annotation. We can distinguish different annotations by different IDs.
- *Name* provides a reference for domain experts to understand or modify annotations. The name attribute is repeatable.
- *ImplementationType* indicates the classification of annotations, i.e., *ProcessType*, *LocalType*, and *MessageType*. The first two types are activity-based annotations triggered before or after an activity. The last one, *MessageType*, is a global annotation type in the PAIS. With *MessageType*, a predefined event will trigger corresponding processing at any point during the execution of a process. One example is processing a real-time event message from logistics in our case studies.
- *Ifittt-rules* is a set of IFTTT style rules. Domain experts use them to authorize business logic dealing with LTC.

Table 1 Sample properties of basic annotations

Id = "annotation"
Name = "ForShipping"
ImplementationType = "MessageType"
ProcessDefinitionId = "pid"
Ifittt-rules = [{if:delay-judgment(600), then:delay-topic, url:localhost:9001/IOT/temperature} ,...]
ExternalVariables = [(Vname, Value),...]
ProcessVariables = [(Vname, Value),...]
Policy = "TimePriority"
ValueExchange: [(name:AR2AP), (input,AR), (url, localhost:9002/VDML)]
Type: "AP"
Constraints: [AP < 480, ...]

The URL field in the attribute is optional but useful when we want the help of a third-party service to parse and process annotations.

- *ProcessDefinitionId* relates an annotation to the process model being adapted. This attribute is automatically filled when the process model file is deployed to the process engine.
- *ProcessVariables* is a set of process variables required by IFTTT rules. It is an optional.
- *ExternalVariables* declares additional (beyond process variables) variables in IFTTT rules logic. It is an optional attribute.
- *Policy* specifies a set of policies for a coordinator to choose in coordinating inter-PAIS communication.
- *ValueExchange* describes the transforming method used in value modeling and controlling during inter-process collaborations. It equips BPMN with the capability of value proposition, value exchanging, and value transforming. The main three attributes are the converter name, input parameters, and URL of third-party API. We will deliberate them later.
- *Type* is a name of the business value with which domain experts are concerned. It is important to note that the value naming must be unique within a swim lane in BPMN.
- *Constraints* is a set of rules specified by domain experts. Any execution must meet these constraints; otherwise, an exception is thrown out.

Each annotation contains the above essential attributes except those marked as optional. Now let's investigate the lifecycle of annotations and the structure of the two types of them, i.e., Activity-Annotation and Message-Annotation.

4.2 Life cycle of annotations

Annotations have their lifecycles, depending on the type of annotation. This subsection describes the two kinds of them, i.e., when they are activated, how to run the process, and how to terminate it.

Activity-annotation is an annotation decorated on an activity of a process model. There are two dimensions to classify Activity-annotations, i.e., pre- or post-annotations related to an activity and local or process-scale ones related to the execution logic. A pre-annotation starts just before the execution of the addressable activity. Its result will affect the execution of the addressable activity. In contrast, a post-annotation begins immediately after the execution of the addressable activity and accepts all results of the addressable activity.

Following the other dimension, the implementationType field of an annotation specifies it as the LocalType or ProcessType. For the LocalType annotation, the process engine first pauses the execution flow, enters a local environment, and evaluates the IFTTT rules in the annotation's ifttt-rules field. Whenever the local evaluation exits, the process engine finishes the annotation's lifecycle and resumes the original execution flow of the process. However, suppose the annotation is a ProcessType. In that case, the process engine will generate internal events, which will trigger the *event capturer* in Fig. 1. Then, the internal events will be packaged as external events for communication and interaction with the external world. Figure 2 summarizes the whole story.

A Message-Annotation is related to a whole process model and can be triggered at any moment during the execution of the process. When this annotation is activated, its lifecycle is almost identical to that of ProcessType in Activity-Annotation, except that the process execution flow never needs to suspend.

5 Message routing mechanism

In contrast with the case of the Activity-Annotations, Message-Annotation is triggered by a message. Usually, these messages represent some external events outside the PAIS in concern, such as a temperature value or the delay of a vessel. Message-Annotations are handlers in responding to these events. The message routing mechanism is designed to bridge external events to Message-Annotations. When a message outside the PAIS comes in, it is first processed by the

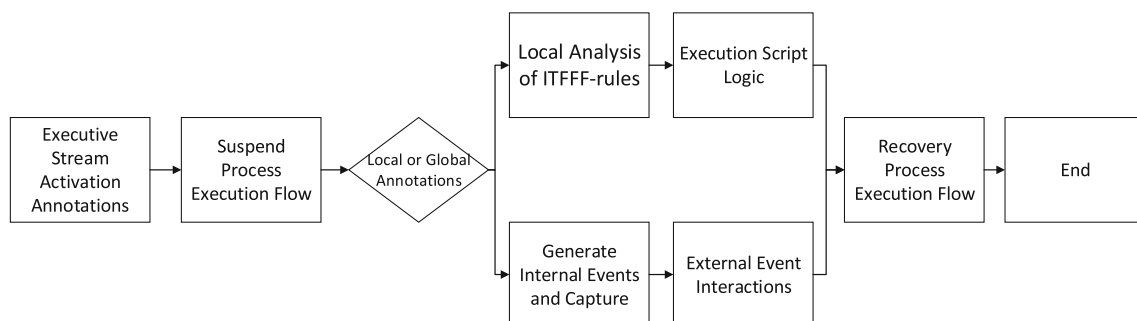


Fig. 2 Lifecycle of activity-annotations

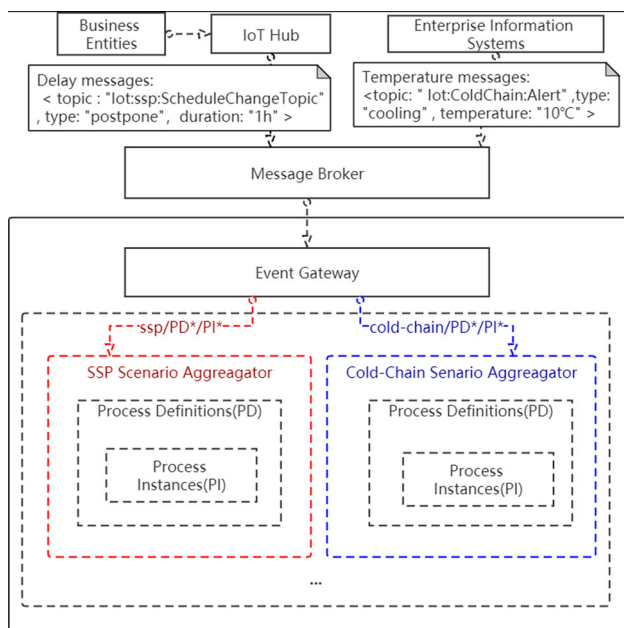


Fig. 3 Overview of the message routing with two scenarios, i.e., the SSP problem in red and the cold chain transportation in blue (Color figure online)

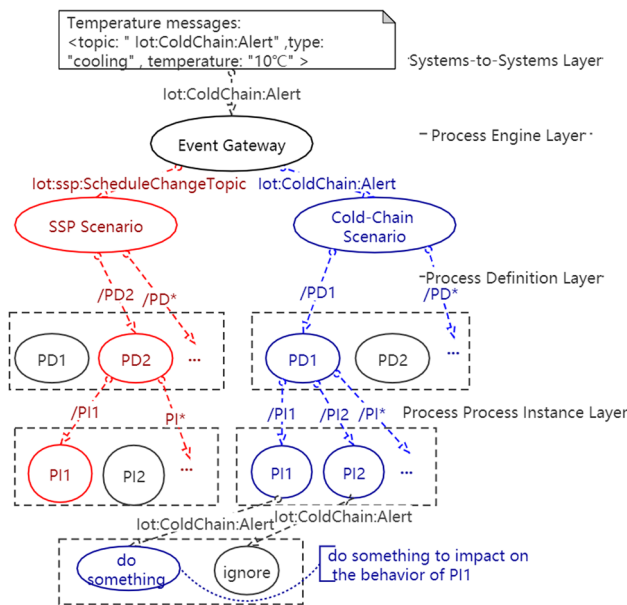


Fig. 4 Message routing in the example process

message routing mechanism before being sent to a specific process instance in triggering a Message-Annotation.

The *Event Gateway* in Fig. 1 serves as the message routine mechanism. It relies on the support of *Private Pub/Sub Hub* component that deals with environment events and the *Context Sharing* component that facilitates the inter-PAIS communication. Both components take the publish/subscription messaging pattern.

Figure 3 illustrates the overall message routing framework with two sample message sources. All works are around top-

ics, and all annotations attached to the addressable model have been transformed into corresponding objects stored in the annotation manager during the model parsing phase. The external message routing process extracts information from the annotation object, especially the topic field, and then uses it to complete the entire external message routing activities (Fig. 4).

- *Message broker for system-to-system*: External messages come from IoT information centers or other enterprise information systems (EIS), and the system subscribes to topics on these sources. For example, a message of temperature from IoT:

```
< topic: "lot:ColdChain:Alert",
  type: "cooling",
  temperature: "10°C" > ,
```

or the delay message from an EIS on a vessel:

```
< topic : "lot:ssp:ScheduleChangeTopic",
  type: "postpone",
  duration: "1h" > .
```

We use the Advanced Message Queuing Protocol (AMQP)² to handle messages from all scenarios, and the RabbitMQ³ treats messages equally regardless of the source.

- *Event gateway at the process engine layer*: The event gateway is responsible for authenticating which application scenario the external message is acting on. The topic field in a message indicates the scenario in which the external message is consumed. The correspondence relationship between the topic and the addressable scenario could be one to one or one to many. Depending on the content of the topic field and the scenario relationship stored in the system, the *Event Gateway* decides whether the external message is for a specific scenario represented by a leaf node or a scenario described by a non-leaf node. In the latter case, the event gateway will broadcast the message to all scenarios as the children of that non-leaf node. However, the correspondence between process instances and application scenarios is not defined in advance in the Event Gateway. The *scenario aggregator* takes care about the mapping between process definitions, process instances, and application scenarios.
- *Scenario aggregator at the process definition layer*: As a process model is deployed, annotations attached to the model are transformed into memory as objects that the

² <https://www.amqp.org/>.

³ <https://www.rabbitmq.com/>.

annotation manager manages. The scenario aggregator scans the Message-Annotation object in the annotation manager and establishes a one-to-one correspondence between the application scenario and the process definition ID based on the topic field in the IFTTT rules attribute, where the content of the topic field determines the scenario. Before the process is instantiated, the scenario aggregator has already maintained an application scenario: a process-defined relationship table, such as

```
[scenarioA : [PDID1, PDID2 . . .],
  scenarioB : [PDID3, PDID4 . . .], . . . , ]
```

When the process engine starts the model, it generates a new process instance. This instance will be registered in the scenario aggregator, e.g.,

```
[scenarioA : [PDID1 : [PIID1, PIID2],
  PDID2 : [PIID3, PIID4], . . .]]
```

Once the process instance ends, the scenario aggregator deletes this record accordingly.

- *Interested message selector at the process instance layer:* At the process instance level, instances can also select whether it cares about messages. The Message Selector (IMS) is responsible for identifying the addressable process instances (PIs) interested in external messages. The IMS helps each PI maintain a collection of interesting messages in the runtime phase. The functionality is realized with a RESTful API. This component is designed with the intention that
 - A model may have multiple Message-Annotations,
 - There may be many scenarios associated with it,
 - It will be useful in real production environments where immediate changes may result in scenarios that do not need to be executed or where execution will result in program errors.

6 Design of a DSL-Anno

This section deliberates on how to design a DSL, Anno.

6.1 The construction of the abstract syntax tree

The abstract syntax tree (AST) is a tree aimed at representing the conceptual syntactic structure of the source code [14]. The design of the AST in this paper is based on the generative requirements of the addressable code. In our setting, the addressable code comprises three aspects, i.e., functional properties, non-functional properties, and value exchange. The functional attribute refers to functionality, e.g., checking the value of the attribute obtained. The non-functional

attribute usually relates to QoS metrics such as reliability, cost, availability, and response time, as well as quality constraints (e.g., yield > 80%), restrictions on values (e.g., net revenue > \$100,000), and business objectives (e.g., maximizing user satisfaction). In contrast, the value exchange concerns the business value delivery by a value exchange.

We propose the IFTTT [2] rules as the cornerstone for building the code framework, extended with variable definitions/invocations, service definitions/invocations, topic definitions/invocations, and attribute value comparisons as the elements that populate the IFTTT rules. Thus, the three aspects of the addressable code can be accomplished by IFTTT rules at different levels. Figure 5 shows the AST formed from the keywords extracted from an addressable code. The AST expands down from the root node, and each layer is a token belonging to the same level, from the definition of topics, functions, etc., all the way down to the meta-characters that topics and functions use. The AST is designed to meet the development needs of domain experts.

6.2 The implementation of the editor

We use the Eclipse XText⁴ to build the syntax-directed editor. It consists of two aspects, the use of AST, and the description of the syntax rules.

The Ecore model is used in constructing the AST. After importing the Ecore model into Eclipse, it will automatically retrieve the corresponding Tokens from the EClass, EAttribute, EReference, and EDataType described in Ecore. The device also serves as guidance for the development of DSLs. DSL developers must work following this AST. Otherwise, parsing and compilation errors will arise. This design ensures that the target DSL is coherent with the syntax rules.

Domain experts specify the required syntax rules and conventions in the DSL according to the requirements and keywords of the AST. There are three main levels of syntax descriptions, i.e., the keywords used in the DSL, the letters/numbers/special characters used in the custom variables, and the keywords or variables used in combination to define the required syntax rules.

Whenever a token appears in the editor but not in the AST, the editor throws an exception with the message “cannot create the type for Test because its EPackage is unknown”. This design maintains a good balance of DSL correctness and the reuse of generators.

6.3 The implementation of the generator

Both the generator and the grammar design are all based on the AST technology.

⁴ <https://www.eclipse.org/Xtext/>.

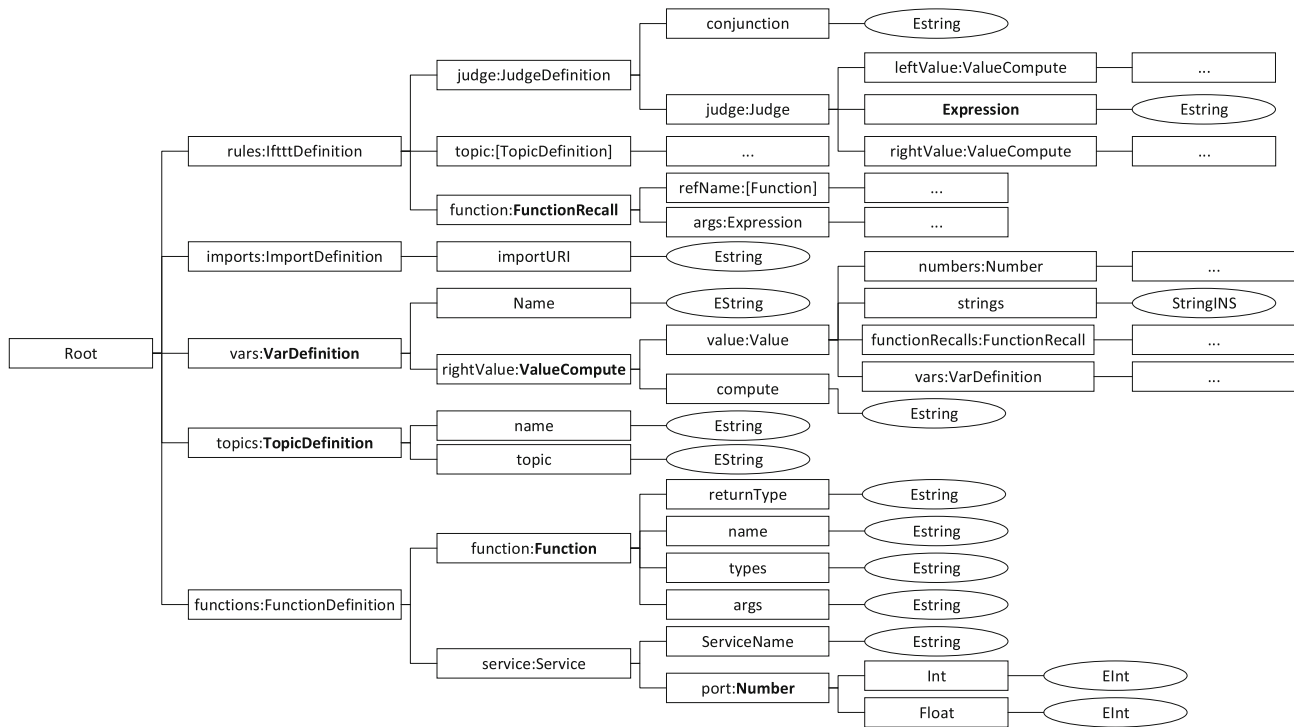


Fig. 5 Abstract syntax tree defined by addressable code

Fig. 6 Anno's grammar definition

```

anno → IF boolean_exp THEN actions
boolean_exp → TRUE | FALSE | and_exp | boolean_exp || and_exp
and_exp → and_exp && equality_exp | equality_exp
equality_exp → equality_exp equality_oprelational_exp | relational_exp
equality_op → == | !=
relational_exp → relational_exprelational_op additive_exp | additive_exp
relational_op → < | <= | > | >=
additive_exp → additive_exp add_op multiplicative_exp | multiplicative_exp
add_op → + | -
multiplicative_exp → multiplicative_exp mult_op unary_exp | unary_exp
mult_op → * | /
unary_exp → (unary_op primary_exp) | primary_exp
unary_op → + | -
primary_exp → (boolean_exp) | constant | service_call
constant → number | string
number → nz_digitdigit
nz_digit → |1| ... |9|
digital → 0 | nz_digit
string → "character {character | digital}"
character → |a| ... |z| | |A| ... |Z|
service_call → valid_url_with_parameters
action → Topic string
valid_url_with_parameters → return_type url_name parameter{parameter}
return_type → number | string
function_name → string
parameter → string | number

```

Code parsing generally goes through three steps, i.e., lexical analysis, syntax analysis, and code generation. We use tools in Eclipse to extract Token and AST construction of the former two steps and use AST for the code generation.

In code generation, the defined variables need to be analyzed to turn indirect references into direct ones. AS reading the code line by line, variable definitions and invocations are steadily updated so that the variable names stored in the table are up to date with the values stored.

As shown in Algorithm 1, there are four main types of right values in the equation right value analysis algorithm, namely, number types, string types, functions, and variable name references. Different types of right values need different processing types; for numbers and strings as a constant type, just directly stored in the control table, but for function types, it needs to read out all the contents of the function and save them in the control table. Finally, it needs to read out the corresponding values from the control table to store the variable name for variable name references. According to the calculation, the time complexity of Algorithm 1 is $o(1)$ at best, $o(n)$ at worst, and the space complexity is $o(1)$. Based on the time and space complexity analysis already average code size shows that the number of equations in the whole code is finite. It takes about 0.5 milliseconds on average to run an analysis of the right value of an equation once, so Algorithm 1 is guaranteed to run the program to completion within the acceptable range of the user.

6.4 The implementation of Anno

Now, it is time to use the generator to produce a syntax-directed editor for a particular case. To conduct case studies in 7, we customize an AST and make the annotation-editor Anno, as shown in Fig. 7. The design of Anno follows the top-

Algorithm 1: Analysis of the right value of an equation

Input: *value* right value, *vars* table of left and right values of the equation

Output: *result* direct reference to the right value

```

1 if value.num != null then
2   | result ← value.num.getInt() + "." + value.num.getFloat() ;
3 else if value.str != null then
4   | result ← value.str ;
5 else if value.function != null then
6   | result ← value.function.functionName + "(" ;
7   | count ← 0 ;
8   | for arg ← value.function.getArgs() do
9     | if count != 0 then
10    |   | result ← result + "," + arg.getInt() + "." +
11    |   |   | arg.getFloat();
12    |   | else
13    |   |   | result ← result + arg.getInt() + "." + arg.getFloat();
14    |   | count ← count + 1;
15   | result ← result + ")";
16 else if value.varCall != null then
17   | result ← vars.get(value.varCall);
18 return result

```

down manner. First comes the IFTTT framework, roughly structured as *IF* followed by Boolean expressions, *THEN* followed by the action accordingly. If the condition stands, the process continues to run. Otherwise, it has to respond in some way, such as just ending the entire process, throwing some message, etc. Next, Boolean expressions and actions need to be specified. Boolean expressions mainly include elementary propositions valued with TRUE or FALSE, combined with operators AND and OR. The action part consists of a topic and a function. The topic part represents some message, and the function part invokes some services. The last task is the specification of elementary boolean expres-

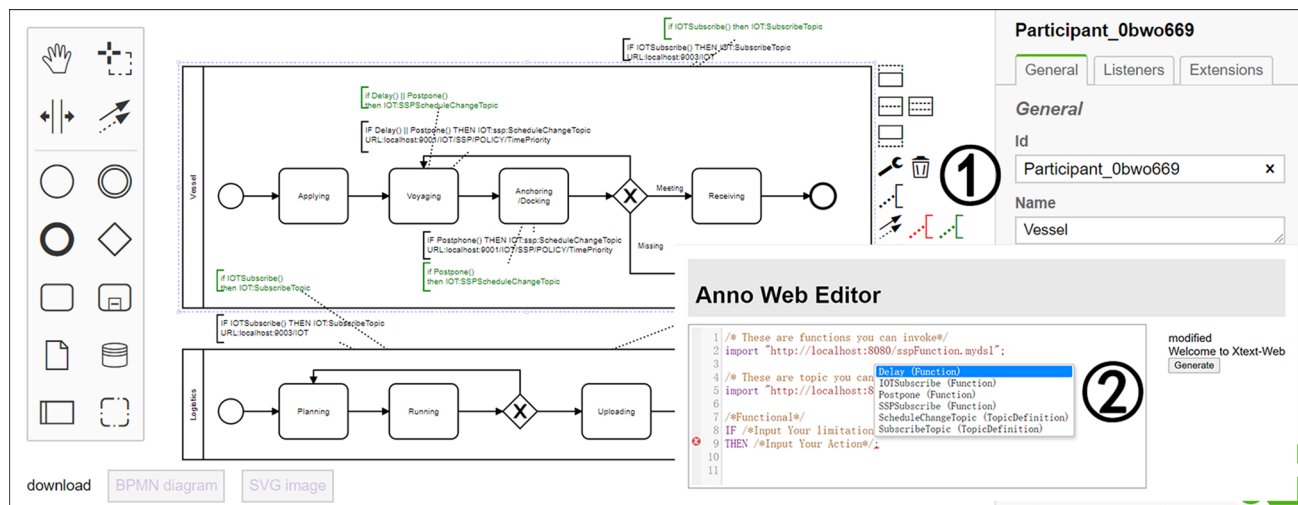


Fig. 7 A screenshot to illustrate the modeler and the syntax-directed editor Anno

sions, represented using basic letters, numbers, and special characters.

The entire grammar follows the IFTTT style. To conduct case studies in 7, functions and topics borrow corresponding settings in [15]. All the designs facilitate the definition of variables from the user experience point of view. The domain expert does not need to consider the type of object to be obtained and defined.

In a word, our approach takes advantage of the highly abstract nature of ASTs to build generic generators. Then, the generator can produce a specific syntax-directed editor with a target syntax convention specified by domain experts with particular scenarios. For example, Anno is such a customized editor for the SSP problem in case studies.

7 Case study

7.1 The business process model

The model shown in Fig. 9 is the business process of a vessel in the classic ship spare parts (SSP) replenishment business. The SSP characterizes the ability of a voyaging vessel to receive spare parts from ground logistics at a future port on its shipping route. The vessel has consumed spare parts on board to fix some faults during the voyage. In worrying about failure in the future journey, the vessel applies for replenishment of spare parts. Upon receiving the application, the shipping company's manager will place an order with the supplier, who will notify the logistics company to deliver the spare parts to the vessel. The business is straightforward if there are no long-tailed events that force the behavior of both vessel process and logistics.

Usually, the industry needs multiple parties to solve the problem of supplying spare parts, and three new players also exist in this business. Figure 8 depicts the overall communications in SSP. Once the request for spare parts is passed to the logistics company by vessel, management, and supplier, the logistics company will calculate the delivery location of the spare parts according to the vessel's shipping schedule and transport them by road. The logistics company bears the storage and logistics costs during the land transport, which is proportional to the retention time of the goods in the port. Hence, the land logistics prefers to deliver precisely when the vessel arrives (*Just-in-time*). If the vessel comes to the dock early and land logistics have not yet reached, this delivery and land logistics need to be re-routed and delivered to a new port. Since, in the original business, land logistics could only determine the port of rendezvous by looking at the vessel's arrival schedule, it is difficult for land logistics to be notified of any changes in the vessel's actual transportation, such as delays or advancements, and to re-route the vessel. In addition, the road conditions of the land logistics will also affect

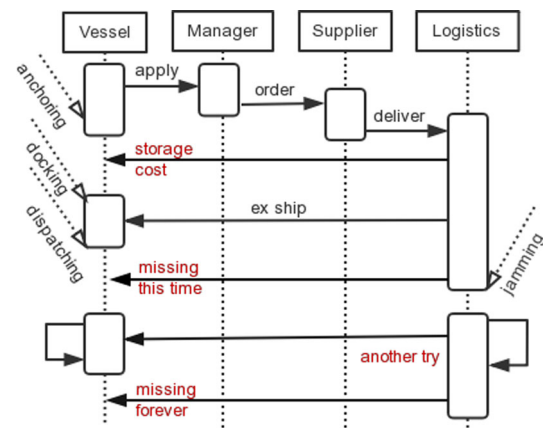


Fig. 8 Message flow in the SSP (Ship Spare Parts) replenishment problem [13]. Long-tailed changes (LTCs) arose from sporadic and irregular events denoted as dotted arrows on both sides. If these LTCs are not handled well, they will violate functional and non-functional constraints marked in red (Color figure online)

the delivery. The vessel cannot change its shipping plans alone because of the land logistics, which may also result in a missed delivery. When the number of missed deliveries is too high due to practical constraints such as the vessel arriving at its destination or the road conditions in land logistics being too disruptive to catch the vessel, the delivery fails, and the entire business process ends.

As the advance of the IoT (Internet of Things) [16], the SSP embraces a new approach that its participants can negotiate as they are operating in their ways. One new requirement arises, i.e., how to reuse legacy business process models to the maximal extent. Enterprises' business process models are knowledge wealth tested by long-term practice. Building a new set of business process models for this change will inevitably bring many risks and challenges to the enterprise. By incorporating IoT capabilities properly, shipping companies can deal with spodic events smartly. Equipped with the process adaptation mechanisms proposed in this paper, the enterprise can use annotations on the existing business process model without hurting the original business.

As can be seen from the annotations (the green snippets) in Fig. 9, the IoT system provides timely information such as speed, position, temperature, and humidity. Some of these topics can help the logistics company adjust its behavior by subscribing to the vessel's particular topics and making decisions accordingly.

In the SSP instant decision scenario, the process needs to have the ability to sense external disturbances (delay or advance) and send message events to an external coordinator, which notifies the land wagon process, recalculates the route, and agrees on a rendezvous port. The added annotations are shown as green snippets in Fig. 9 for the shipping and land transport models, which subscribe to the SSP scenario's delay information via Activity-Annotation and Message-

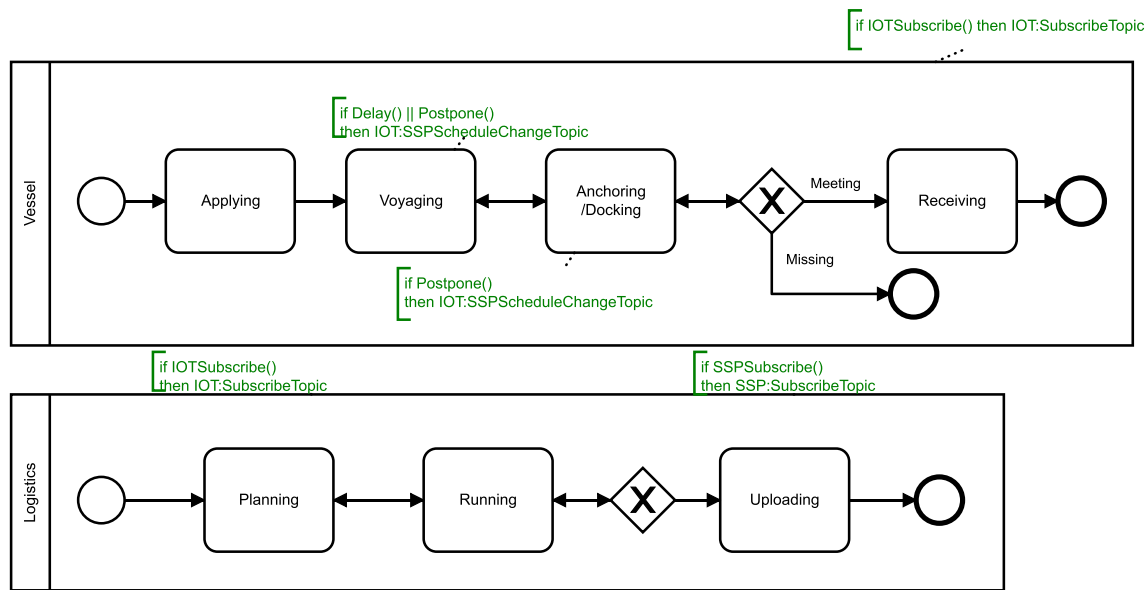


Fig. 9 A sample business process model with annotations (in green) for the SSP problem (Color figure online)

Annotation, respectively, where Activity-Annotation is an annotation attached to a specific activity of the process model. Such annotations may only be activated when that attached activity is active.

7.2 Comparative analysis of transport costs

To validate the effectiveness and efficiency of our solution in this subsection, we conduct a qualitative and quantitative analysis of the SSP problem. The qualitative one is straightforward because changing the behavior of both a vessel and wagon by annotations can be easily achieved with timely messages from IoT. The land transport process can respond instantly to make a decision when there is a delay or advance in the vessel transportation process, etc. Next, we only carry out qualitative validation for the scenario. More specifically, we will analyze the improvement in some KPIs (Key performance indicators) in SSP by contrasting the real-time decision scenario with the original processes.

We define the following variables to facilitate the calculation of transport costs during delivery:

- i : The serial numbers of the available ports, for the convenience of experimentation this paper selects four ports that will stay in the actual business scenario as experimental ports, from 1 to 4 they are Wuxue, Anqing, Nanjing and Zhenjiang.
- $T_{i|L}$: The moment the vessel departs from the port i .
- L_i : The latitude and longitude of port i .
- ΔT : Delay of the vessel.
- S_i : The cost of storage per unit of time at port i , defaults to one unit of weight since the weight of transported goods is not meaningful in this experiment.

- (T_{ilt}, L_i) : Representing, respectively, the moment of arrival of the wagon at port i and the geographical location of the wagon at that moment.
- $F(L_t, L_i)$: The freight cost from the wagon's current location to the port of i is dynamically calculated by obtaining the geographic distance from the AutoGeo Maps API.

If the wagon wants to be delivered at port i , it needs to meet the condition $T_{i|L} + \Delta T \geq T_{ilt}$, i.e., the wagon needs to arrive before the vessel leaves the port. Set Cost_i to be the cost of the wagon coming at the port delivery of i . It is related to the cost of storing the cargo at the port and land transportation cost and is calculated as follows.

$$\text{Cost}_i = \max \{T_{i|L} + \Delta T - T_{ilt}, 0\} \times S_i + F(L_t, L_i) \quad (1)$$

For each port to be selected, the cost of transportation can be calculated according to Eq. 1 if there is no delay or advance events, and again according to ΔT if there is a delay or advance event.

Table 2 is a compilation of experimental data at the time of one delay, in which the original delivery port was the port Anqing. From the data in the table, we see that in the absence of a delay, the wagon chose the port Anqing as the lowest cost destination because the wagon could not reach the port Wuxue for delivery before the vessel's departure. When a delay occurs, the costs of transportation for each port change. The logistics cost increases because the logistics party needs to rent a warehouse longer. Meanwhile, a new opportunity emerges to adjust a new delivery port based on these logistics costs. To analyze the cost variations, we define three indicators:

Table 2 Transport costs by port before and after the delay

Port	Wuxue	Anqing	Nanjing	Zhenjiang
Pre-change logistics costs (RMB)	Undeliverable	3913.98	4628.00	4595.62
Post-change logistics costs (RMB)	3833.96	4096.26	4906.79	5276.22

- Cost change before and after the change occurs, i.e., the rate of change in logistics costs before and after the change happens,
- Cost change before and after the corresponding decision, i.e., the rate of change of logistics costs for the newly selected delivery port after the change is calculated based on logistics costs for the original port after the change, and
- Cost change before and after the change decision, the rate of change of logistics costs for the new port after the decision is calculated based on logistics costs for the original port before the change is calculated based on logistics costs for the new port after the change.

From the table, we can find that if the change had been made without making a new decision on the port of delivery at Anqing, the cost of transportation would have increased by 4.66%, even if Anqing had been the original port of delivery. However, if the decision had been made, it would have been 6.06% lower than if the change had occurred, and even the logistics costs would have been 4% lower than if the new port had been decided before the change. It can be seen that if the process can be adapted to the SSP immediate decision scenario, not only does it reduce the additional cost of the change for all process participants, but if certain conditions are met, the participants can seize the change and receive a higher benefit than if the change had not occurred.

7.3 Analysis of the benefits of the editor

Next, we evaluate the Anno, a DSL shown in Table 3, using a designed abstract syntax tree and adapts the business process model for SSP using the methods proposed in this paper. As shown in Fig. 9 at first glance, we can see that the green annotations are much domain experts friendly, easier than remodeling the whole business process for a sporadic LTC, and effectively hiding the generated object code in a model file.

More precisely, we investigate Anno's use by 20 people from different levels, including programmers and business analysts. To reduce the error in the measurement, we take the average value for ten instances of users' use of the editor. Table 3 shows a comparison between the user writing annotations supported by the engine and user defined, both of which implement the verification of the distribution and ordering places and respond to events. When representing

event nodes with the same meaning, we found that the amount of code written with syntax-directed editors in annotations is much smaller, more straightforward, and more precise than the code written by adding annotations directly to the modeler. In this respect, it is also clear that adding annotations using a syntax-directed editor is better than adding it directly to the modeler.

Further, we compare the effort of using Anno versus directly adding annotations to the modeler in terms of code volume, the average error rate of coding (Eq. 2), and the average time of coding (Eq. 3). Here, the code volume refers to the number of lines of code (LoC) written when modifying the model. The average time of coding and the average error rate of coding are calculated by investigating the length of time it takes to write the code when the user adds annotations and the number of errors in the written code, respectively. LoC measures the amount of code that needs to be written. The average writing time measures the time it takes a user to organize the same logic and write the code. The average write error rate estimates the likelihood that a user will make an error after organizing the same logic and writing the code.

$$\text{AERoC} = \frac{e}{s} \quad (2)$$

$$\text{AToC} = \frac{t}{s} \quad (3)$$

where e represents the total number of errors coded by those participating in the survey, t represents the total number of hours required to code by those participating in the survey, and s represents the total number of people participating in the survey.

We apply these measures to the SSP case. The baseline is to add annotations without the help of Anno. From the results shown in Table 4, we can find that every performance metric using Anno is far superior to adding annotations directly to the modeler.

It is worth mentioning that using a DSL editor based on AST can play a more significant role for different applications or scenarios with different temporal or spatial situations for the same application.

8 Conclusion and future work

This paper focuses on enabling enterprises to efficiently manage business processes in a long-tailed change (LTC)

Table 3 Syntax supported by the engine versus user-defined syntax

Syntax supported by the engine	User-defined syntax	Explanation
IF Delay() Postpone() THEN IoT:ssp:ScheduleChangeTopic URL:localhost:9001/IoT/SSP/POLIC	if Delay() Postpone() then SSPScheduleChangeTopic;	Notification of change of itinerary in case of delay or postpone of itinerary during the voyage
IF Postphone() THEN IoT:ssp:ScheduleChangeTopic URL:localhost:9001/IoT/SSP/POLICY/TimePriority	IF Postphone() THEN SSPScheduleChangeTopic;	Notification of change of itinerary in case of docking delays
IF IoTSubscribe() THEN IoT:SubscribeTopic URL:localhost:9003/IoT	if IoTSubscribe() then SubscribeTopic;	Vessel receives IoT notification if subscribed to IoT
IF IoTSubscribe() THEN IoT:SubscribeTopic URL:localhost:9003/IoT	if IoTSubscribe() then SubscribeTopic;	Logistics company receives IoT notification if subscribed to IoT
IF SSPSubscribe() THEN SSP:SubscribeTopic URL:localhost:9001/IoT/POLICY/TimePriority	if SSPSubscribe() then SSP:SubscribeTopic;	Logistics company receives IoT notification from vessel if subscribed to vessel's IoT

Table 4 Evaluation results for two different ways of adding annotations

The way to add an annotation	LoC	AToC (s)	AERoC (%)
Using the syntax supported by the engine	15	67.3	18.75
Using the customized syntax	10	33.1	0.00

LoC donates the line of code, AToC donates average time of coding, AERoC donates average error rate of coding

environment. As the long-tail theory reveals, in today's highly changing social and technological conditions, the impact of events within the long-tail region on business will not be lower than that of high-frequency events in the short-head area. The same is true for the maintenance of business process models. The approach proposed in this paper helps enterprises manage business processes efficiently in the LTC environment by making it easy to reuse the enterprise business process model with annotations and by uncovering the potential value attributes of business interactions and reflecting them at the model level. Meanwhile, it solves the error-prone problem of adding grammar that can be parsed by the engine directly, extracts the abstract grammar tree for the above grammar that can be parsed by the engine, and users can design their own familiar grammar by keywords on the abstract grammar tree and generate user-defined grammar-guided editor generator for users. Ultimately, it achieves fast, low-cost, and correct response to long-tailed changes, maintains the interests of all business parties in the changes, and optimizes the overall business goals. Grammar by keywords on the AST and generate user-defined grammar-guided editor generator for users. The benefits yielded by the proposed method will become more prominent with the increase in LTC scenarios or the range of change types.

In the future, this study will integrate more programming languages as languages for adding annotations, such as Groovy, and Ruby, to provide more options for people with different needs for language use. At the same time, using a scripting language that the process engine itself can parse as a language for adding annotations will also bring many ben-

efits in terms of the security of the process engine. There is no need to invasively modify the process engine, increasing the safety of the process engine itself. Furthermore, from the parsing of the scripting language by the engine, the upgrade of the process engine does not require another intrusive modification of the process engine again, which will reduce the workload considerably.

Acknowledgements The work is supported by Chinese National Key R&D Program (No. 2017YFB1400604).

References

1. Weske M (2019) Business process management. Springer, Berlin
2. Tibbets L, Tane J. "Ifitt," [EB/OL], <https://ifitt.com/>
3. Van Der Aalst WM, Basten T (2002) Inheritance of workflows: an approach to tackling problems related to change. *Theoret Comput Sci* 270(1–2):125–203
4. Casati F, Ceri S, Pernici B, Pozzi G (1998) Workflow evolution. *Data Knowl Eng* 24(3):211–238
5. Reichert M, Rinderle S, Kreher U, Dadam P (2005) Adaptive process management with adept2. In: *Proceedings of ICDE'05*. IEEE, pp 1113–1114
6. Reichert M, Weber B (2012) Enabling flexibility in process-aware information systems. Springer, Berlin
7. Song W, Jacobsen H (2018) Static and dynamic process change. *IEEE Trans Serv Comput* 11(1):215–231
8. Intrigila B, Della Penna G, D'Ambrogio A (2021) A lightweight BPMN extension for business process-oriented requirements engineering. *Computers* 10(12):171
9. Xu W, Su J, Yan Z, Yang J, Zhang L (2011) An artifact-centric approach to dynamic modification of workflow execution. In: *Proceedings of OTM'11*. Springer, pp 256–273
10. Fowler M (2010) Domain-specific languages. Addison-Wesley, Boston

11. D'Ambrogio A, Paglia E, Bocciarelli P, Giglio A (2016) Towards performance-oriented perfective evolution of BPMN models. In: Proceedings of TMS-DEVS'16. IEEE, pp 1–8
12. Bocciarelli P, D'Ambrogio A, Giglio A, Paglia E (2016) A BPMN extension to enable the explicit modeling of task resources. In: Proceedings of CIISE'16, pp 40–47
13. Ye L, Zhu B, Hu C, Zhang L, Truong H-L (2019) On-the-fly collaboration for legacy business process systems in an open service environment. In: Proceedings of IEEE ICWS'19. IEEE, pp 393–397
14. Baxter ID, Yahin A, de Moura LM, Sant'Anna M, Bier L (1998) Clone detection using abstract syntax trees. In: Proceedings of ICSM'98
15. Zhu B, Chen X, Ye L, Zhang L, Truong H-L (2019) Coordinators desing for instant cross-enterprise collaboration. In: Proceedings of CCF ICSS'19
16. Meyer S, Ruppen A, Magerkurth C (2013) Internet of things-aware process modeling: integrating IoT devices as business process resources. In: Proceedings of CAiSE'13. Springer, pp 84–98

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.